Duplicate Pull Requests from Coding Agents: A Live-Queue Evaluation of 66 Detection Strategies

Harshul Jain, Saurabh Yergattikar, Tanya Sah
Independent Researchers

***Abstract*—Coding agents now open pull requests (PRs) at scale, and many of them repeat work that is already open: in one hand-labelled sample, 23.7% of failed agent PRs were duplicates [1]. Duplicate-PR detectors were built for human PRs and scored against random pairs of PRs. We ask how well they work on agent PRs when scored the way they are used: at the moment a new PR is opened, against the PRs actually open in that repository (its *live queue*). From AIDev [2] we build 1867 live-queue duplicate pairs (342 between different authors) and 2965 PRs without a duplicate, and compare 66 strategies: text, code-overlap, link, embedding, existing duplicate-PR bots, learned combinations, cross-encoders, and LLM judges (Qwen2.5 1.5B–14B, Claude Haiku 4.5). A cheap learned combination of text, code, link, and timing signals ranks the true duplicate first for 0.901 of cross-author pairs, against 0.810 for an INTRUDE-style detector and 0.778 for the best existing bot, with 2.5% versus 4.6% false alarms, in 22 ms per PR on a CPU. No LLM judge or re-ranker improves on it. None of the 65 strategies beats the classic detector on Recall@5 with a confidence interval above zero, so our pre-registered success criterion fails. Random-pair scoring ranks strategies differently from live-queue scoring (Kendall $\tau = 0.75$), and duplicates that share no files remain largely unsolved.**

I. Introduction

AI coding agents such as Codex, Copilot, Devin, Cursor, Claude Code, and Jules open hundreds of thousands of PRs on GitHub [2]. Studies of rejected agent PRs list duplicate or superseded work among the main reasons: 142 of 600 failed agent PRs (23.7%) in Ehsani et al. [1], and "resolved by another PR" is the top reason for unmerged agent fix-PRs in Alam et al. [3] (see also [4], [5]). Every duplicate costs a maintainer a review.

Duplicate-PR detection is an old problem [6], [7], [8], [9], [10], [11], but all prior detectors were evaluated on human PRs, and most against *random* negative pairs (e.g., 100,000 random pairs in INTRUDE [7]). A deployed detector faces a different task: when a PR is opened it must rank the PRs open at that moment and decide whether to alert. Work on duplicate bug reports showed that such unrealistic set-ups overstate results [12]. Agent PRs add new twists: long shared boilerplate, agents that replace their own PRs, and several agents working on the same issue.

We contribute: (1) **LiveQueue**, a benchmark of 1867 duplicate pairs and 2965 no-duplicate queries over 76,209 agent PRs, with each new PR's real open queue (median 40 PRs); (2) a comparison of 66 strategies, including five open duplicate-PR bots and LLM judges, with repository-cluster confidence intervals; (3) a duplicate taxonomy and per-class results; (4) evidence that random-pair scoring misleads method choice; (5) an LLM audit of the silver labels; and (6) all code, results, and the tool, Jinx.

II. Research Questions

RQ1 How well do existing detectors and bots work on agent PRs under live-queue scoring? **RQ2** Do embeddings, cross-encoders, or LLM judges beat cheap combined signals? **RQ3** Which kinds of duplicates remain hard? **RQ4** Does random-pair scoring rank strategies the same way as live-queue scoring?

III. Benchmark

Source. AIDev v5 [2] (CC BY 4.0) contains agent PRs up to 31 March 2026 with comments, linked issues, and per-file patches. **Silver labels.** We match PR comments with a strict pattern that requires an explicit PR reference ("duplicate of #N", "superseded by #N", "in favor of #N", "replaced by #N", "same as #N"); comment-based labels follow DupPR [13] and Galbraith and Greer [11]. The later PR of a pair is the query and the earlier one is the gold duplicate. **Live queue.** A query's queue is every PR in the same repository created before it and still open then (or closed less than seven days earlier). We keep a pair only if the gold PR is in the queue. Other silver partners of a query are ignored when ranking. **No-duplicate queries.** We sample merged PRs from the same repositories with no duplicate-like comment; 2965 have a non-empty queue. **Relations.** 1525 pairs are self-replacements (same author) and 342 are cross-author (228 same agent, 114 different agents). We report cross-author results as the main target: a maintainer cannot spot these duplicates from the author name alone. **Taxonomy.** By code overlap: A code twin ($\geq$50% identical changed lines), B partial overlap, C same files with different code, D no shared files (intent-level). We also report pairs with no link at all (no #ref and no shared issue) and pairs with the same code but a different description. **Label audit.** Claude Haiku 4.5 read each pair (comment, titles, truncated descriptions, changed files) and labelled it duplicate, replacement, subsumed (newer PR includes the older change in larger work), related, or unclear. On 200 random pairs, 79.0% [73.5, 84.5] are duplicate or replacement and 97.5% if subsumed counts; on all 342 cross-author pairs, 84.8% and 96.2%. This is an LLM audit, not human annotation.

IV. Strategies

We group the 66 strategies into families. **Simple**: recency; TF-IDF [6] and BM25 on raw text, template-stripped text, or

TABLE I
CROSS-AUTHOR PAIRS ($n = 342$). FALSE ALARMS ON 2965 PRS WITH NO DUPLICATE.

Strategy	R@1	R@5	MRR	False alarms % [95% CI]
Jinx (ours)	0.901	0.971	0.932	2.5 [1.9, 3.3]
All signals, LR	0.906	0.974	0.935	1.6 [1.1, 2.1]
Jinx + BGE rerank + Qwen-7B	0.901	0.974	0.931	2.8 [2.2, 3.5]
Claude Haiku 4.5 + Jinx	0.898	0.971	0.932	1.8 [1.3, 2.3]
Qwen2.5-14B + Jinx	0.871	0.971	0.917	2.9 [2.3, 3.7]
Claude Haiku 4.5 alone	0.842	0.971	0.902	5.4 [4.6, 6.1]
INTRUDE + time [7], [8]	0.810	0.953	0.873	4.6 [3.7, 5.7]
contrib-guard (bot)	0.778	0.918	0.843	11.7 [8.9, 14.6]
Links only	0.839	0.924	0.882	3.8 [3.0, 4.8]
TF-IDF [6]	0.757	0.921	0.832	9.4 [7.8, 10.9]
BGE-small embedding	0.763	0.915	0.830	6.4 [5.3, 7.5]

TABLE II
RECALL@1 PER RELATION AND DUPLICATE CLASS (CROSS-AUTHOR UNLESS NOTED).

Group (Recall@1)	n	Jinx	All LR	INTRUDE+t	c-guard	Haiku+J
same agent	228	0.899	0.908	0.825	0.811	0.895
cross-agent	114	0.904	0.904	0.781	0.711	0.904
A: code twin	139	0.971	0.978	0.957	0.892	0.978
B: partial overlap	164	0.896	0.909	0.762	0.720	0.878
C: same files	10	0.700	0.600	0.400	0.500	0.800
D: no shared files	29	0.655	0.655	0.517	0.655	0.655
no link at all	140	0.821	0.829	0.643	0.664	0.829
self-replacement	1525	0.887	0.892	0.857	0.782	–

the diff. **Code**: changed-file Jaccard, overlapping changed line ranges, identical added lines. **Links**: shared linked issue or a #reference between the PRs. **Embeddings**: MiniLM, BGE-small/base, UniXcoder, CodeBERT, Jina-code on text or diff. **Existing bots**: contrib-guard (two ports), Simili, PRSense, PRism and vector-triage, run as published where possible and otherwise re-implemented with local models. **Learned**: logistic regression (LR) and gradient boosting over INTRUDE-style features plus time [7], [8] ("INTRUDE + time"), over the Jinx features, and over all signals. **Re-rankers and judges** on the top-10 (or top-5) of Jinx: cross-encoders (MiniLM, BGE-reranker base/v2-m3, and a BGE-reranker fine-tuned per fold), and LLM judges that output a yes/no duplicate probability (Qwen2.5 1.5B/7B/Coder-7B/14B on GPU, Claude Haiku 4.5 via API), alone, averaged with Jinx, or stacked with LR. **Jinx** is our tool: template-stripped TF-IDF on title and body, an optional MiniLM cosine, file/hunk/line overlap, links, same author/agent, time gap, and whether the candidate is still open, combined by LR.

V. EVALUATION PROTOCOL

All learned strategies use five-fold cross-validation grouped by repository. For each labelled query we record the rank of the gold PR and report Recall@1, Recall@5, and mean reciprocal rank (MRR). For false alarms we set each strategy's threshold on the training folds at the median top-1 score of cross-author queries and count no-duplicate queries whose top score passes it. Confidence intervals (95%) come from 2,000 bootstrap resamples of repositories. Before running, we fixed a kill rule: *if Jinx does not beat the best baseline on cross-author Recall@5 by a meaningful, CI-backed margin, we say so plainly.*

VI. RESULTS

RQ1: existing detectors. Table I shows that text-only methods are weak on agent PRs (TF-IDF Recall@1 0.757). The best open bot, contrib-guard, reaches 0.778, and INTRUDE + time 0.810. Jinx improves Recall@1 over INTRUDE + time by +9.1 points [+5.0, +13.1] and MRR by +5.9 [+2.7, +9.3], and nearly halves false alarms (2.5% vs. 4.6%). But Recall@5 improves only by +1.8 [-0.9, +5.2]: **the kill rule fails**. Recall@5 is near its ceiling for all combined methods; the differences are in which PR is ranked first. Against contrib-guard, Recall@5 improves by +5.3 [+1.2, +9.7]. On the 290 audit-valid cross-author pairs the Recall@1 gain over INTRUDE + time remains (+7.6 [+3.1, +10.8]) and the Recall@5 gain does not (+0.3 [-1.9, +2.6]).

RQ2: models. Embeddings alone are far behind (MiniLM 0.751, BGE-small 0.763, BGE on the diff 0.515). LLM judges do not help: Claude Haiku 4.5 alone re-ranking the top-5 gets 0.842, averaged with Jinx 0.898; Qwen2.5-7B averaged with Jinx 0.892; the larger Qwen2.5-14B 0.871. The best cascade (Jinx + BGE-reranker + Qwen-7B) ties Jinx (0.901). Using all signals in one LR gives 0.906, within noise of Jinx. The judges add cost ($7.25 for the Haiku run) and latency, whereas Jinx needs 22 ms per PR (p90 229 ms) on a CPU.

RQ3: hard duplicates. Table II shows that code twins are easy: every combined method reaches Recall@1 of at least 0.91. Recall@1 drops on class C (same files, different code; Jinx 0.700) and most on class D (no shared files, $n = 29$; Jinx 0.655, INTRUDE + time 0.517, Haiku + Jinx 0.655). Links explain much of the benchmark's easiness: a links-only rule reaches 0.839. On the 140 cross-author pairs with no link at all, Jinx gets 0.821 and INTRUDE + time 0.643. When the code is the same but the description differs (all relations, $n = 287$), INTRUDE + time beats Jinx (0.826 vs. 0.784); this is consistent with the trained Jinx weights, which give identical changed lines almost no weight once text, files, and links are included.

RQ4: random pairs vs. live queues. Fig. 1 plots each strategy's AUC on random pairs against its live-queue Recall@1. The two rankings agree only partly (Kendall $\tau = 0.75$): 21 of the 25 combined methods have random-pair AUC above 0.98, which makes them look interchangeable, yet their live-queue Recall@1 ranges from 0.851 to 0.906. Fig. 2 shows the operating curves: at every false-alarm budget Jinx and the all-signals LR detect more cross-author duplicates at rank 1 than INTRUDE + time and contrib-guard.

VII. THREATS TO VALIDITY

Labels. Silver labels miss duplicates nobody commented on, so some "no-duplicate" queries may hide duplicates (false alarms are overstated) and some labels are subsumed rather than duplicate. The LLM audit bounds this but is not human annotation. **Link bias.** Commenters often link the duplicate, and the same links are features; we therefore report the no-link subset. **External validity.** One dataset, agent PRs only: AIDev

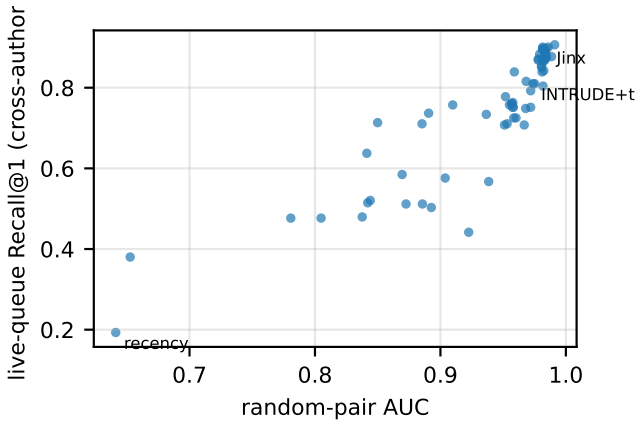


Fig. 1. Random-pair AUC vs. live-queue Recall@1 for all strategies.

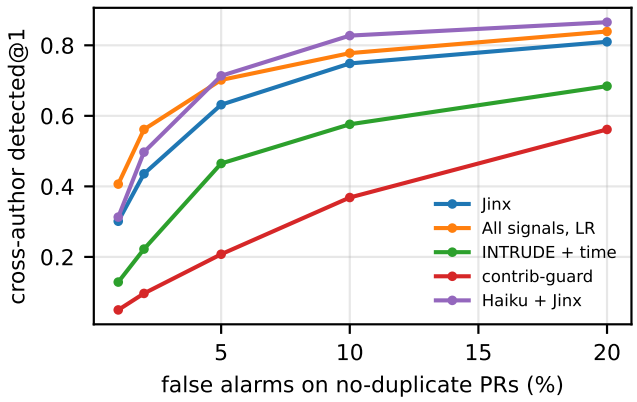


Fig. 2. Cross-author duplicates found at rank 1 above threshold, as a function of the false-alarm rate on no-duplicate PRs.

v5 has no human PRs from the same repositories and time, and our attempt to fetch more PRs failed, so we cannot compare agent and human duplicates. Self-replacement dominates (82% of pairs). **Bots.** Some bots needed re-implementation with local models; our ports may differ from the deployed tools. **Multiple comparisons.** We compare many strategies; we fixed the main comparison and the kill rule before running.

VIII. Related Work

Duplicate PRs. Li et al. introduced TF-IDF detection [6] and the DupPR dataset [13]; INTRUDE added code and issue clues [7]; later work added time [8], graph embeddings [9], LLM summaries [10], and issue–PR linking [11]; a recent review covers the field [14]. Empirical studies examine competing and redundant PRs [15], [16]. None evaluates on agent PRs or against live queues. **Agent PRs.** Studies of agent-PR failures report duplicates and superseded work [1], [3], [4], [5]; others study merge conflicts between co-active agent PRs [17], coordination before a PR exists [18], and references between PRs [19]. **Duplicate bug reports.** Zhang et al. showed that realistic evaluation shrinks reported gains [12]; CUPID combines an LLM with retrieval [20]. We bring the realistic-evaluation argument to PRs.

IX. Conclusion

On agent PRs scored against real open queues, a cheap combination of text, code, link, and timing signals is the best of 66 strategies we tested, beats existing duplicate-PR bots, and ranks the right PR first more often than a classic INTRUDE-style detector with fewer false alarms. It does not clear our pre-registered Recall@5 bar, LLM judges add nothing, and intent-level duplicates with no shared files remain open. Random-pair AUC should not be used to choose a duplicate-PR detector.

Data Availability

Benchmark, code, raw scores, logs, and the Jinx tool: [TODO: Zenodo DOI and repository URL].

References

[1] R. Ehsani *et al.*, "Where do AI coding agents fail? an empirical study of failed agentic pull requests in GitHub," 2026, mSR 2026.
[2] H. Li *et al.*, "AIDev: Studying AI coding agents on GitHub," 2026.
[3] Alam, Mondal, and C. K. Roy, "Why are AI agent involved pull requests (fix-related) remain unmerged?" 2026.
[4] Nakashima *et al.*, "Why agentic-PRs get rejected: A comparative study of coding agents," 2026.
[5] Abujadallah, Arabat, and M. Sayagh, "Understanding the rejection of fixes generated by agentic pull requests: Insights from the AIDev dataset," 2026, tODO: verify author first names.
[6] Z. Li, G. Yin, Y. Yu, T. Wang, and H. Wang, "Detecting duplicate pull-requests in GitHub," in *Proc. Internetware*, 2017.
[7] L. Ren, S. Zhou, C. Kästner, and A. Wąsowski, "Identifying redundancies in fork-based development," in *Proc. SANER*, 2019.
[8] Q. Wang, B. Xu, X. Xia, T. Wang, and S. Li, "Duplicate pull request detection: When time matters," in *Proc. Internetware*, 2019.
[9] J. Jiang, Q. Li, J. Guo, Z. Li, L. Zhang, and G. Li, "DupHunter: Detecting duplicate pull requests in fork-based development," *IEEE Trans. Softw. Eng.*, 2023.
[10] Z. Liao, Y. Liu, Y. Lan, and Y. Sun, "DupLLM: Duplicate pull requests detection based on large language model," in *Proc. APSEC*, 2024.
[11] R. Galbraith and D. Greer, "Detecting duplicate effort in GitHub contributions," in *Proc. ENASE*, 2025.
[12] T. Zhang, D. Han, V. Vinayakarao, I. C. Irsan, B. Xu, F. Thung, D. Lo, and L. Jiang, "Duplicate bug report detection: How far are we?" *ACM Trans. Softw. Eng. Methodol.*, 2023.
[13] Y. Yu, Z. Li, G. Yin, T. Wang, and H. Wang, "A dataset of duplicate pull-requests in GitHub," in *Proc. MSR*, 2018.
[14] B. Chekaya, Garrouch, and Omri, "Duplicate pull requests in code management platforms: A systematic literature review," Research Square preprint, 2026, tODO: verify author first names.
[15] X. Zhang, Y. Chen, Y. Gu, W. Zou, X. Xie, and X. Jia, "How do multiple pull requests change the same code: A study of competing pull requests in GitHub," in *Proc. ICSME*, 2018.
[16] Z. Li, Y. Yu, M. Zhou, T. Wang, and G. Yin, "Redundancy, context and preference: An empirical study of duplicate pull requests in OSS projects," *IEEE Trans. Softw. Eng.*, 2022.
[17] Xu *et al.*, "AI agent pull requests on GitHub: Frequency, structure and merge conflict rates," 2026.
[18] Sarkar, "Before the pull request: Mining multi-agent coordination," 2026.
[19] Khemissi *et al.*, "Humans integrate, agents fix: How agent-authored pull requests are referenced in practice," 2026.
[20] "CUPID: Leveraging ChatGPT for more accurate duplicate bug report detection," 2023, tODO: authors.